

COMP 305 Project Baby Competition

Group 18:

Ali Karataş

Burcu Özer

Melodi Ezgi Keskin

Sedat Çoban



KOÇ
ÜNİVERSİTESİ



Outline

- Problem Definition
- Lemma
- Solutions
 - Complexities
 - Run Times
 - Pseudocode
- Future Work
- Questions



Problem Definition

Inputs:

- Number of babies (n)
- Number of Goal Posts (m)
- Scores of at Each Goal Post

Upper Bound (l) $\rightarrow$ final places for each baby

Outputs:

- Smallest Possible Final Places for each Baby



Solution 0: Brute Force All I Values Between Max and Min

- Travel all of the I values from largest score to smallest score
- Construct minimal final place with respect to each I value
- Gives correct output
- Time Complexity:
 $O(f(n,m) \times (\text{max_score} - \text{min_score}))$
- Space Complexity: $O(n \times m)$



Lemma

- Using values other than the scores as L values would not affect the final minimum placement



Solution 1 - withSet

- Travel all distinct l values
- Construct minimal final place with respect to each l value
- Gives correct output
- Time Complexity: $O(n^3 \times m)$
- Space Complexity: $O(n \times m)$
- Runtime (in Python): 46.717s



Solution 2 - L_Values Array Algorithm

- Augment first solution with two arrays. One of the array keeps the total point of the players. Another array keeps index of the last changed score of the players.
- Gives correct output.
- Time Complexity: $O(n^3 \cdot m)$
- Space Complexity: $O(n \cdot m)$
- Runtime (in C): 4.861s



Pseudocode for Solution 2

- 1) Take input
- 2) Read File into Arrays
- 3) Construct I Values array
- 4) Construct Augmentation Arrays (Sum and Index Arrays)
- 5) Calculate minimal final place
- 6) Travel all distinct I values:
 - a) Travel all players:
 - i) Update sum of the player using values smaller than old I value
 - ii) Update player's score until encounter I value > score of the player
 - b) Calculate minimum final place with respect to updated sums' of the players
 - c) Compare the new found minimal final place with the previous one

Total Complexity: $O(n^3 * m)$.



Graphs for Complexity Analysis

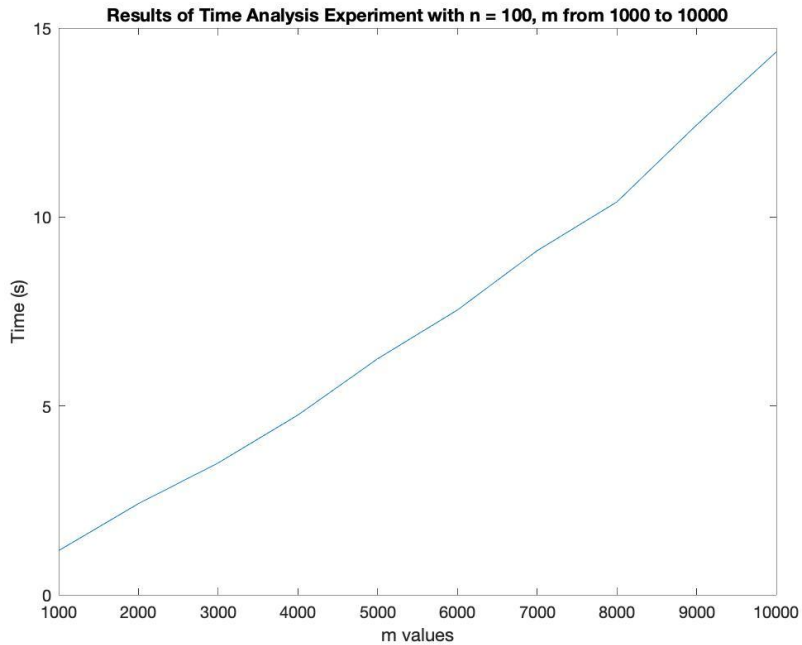


Figure 1: Complexity Analysis for m

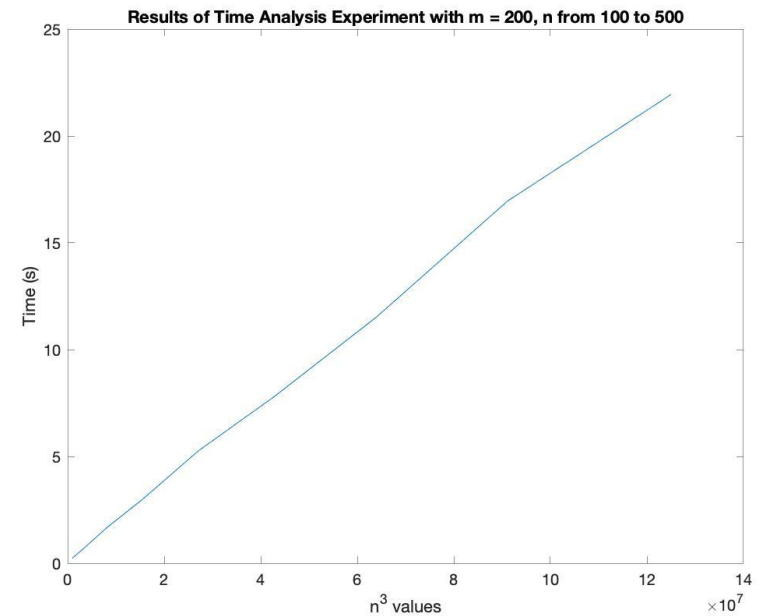


Figure 2: Complexity Analysis for n



Solution 3 - Monte Carlo Algorithm

- The first run is similar to solution 2
- For each L value, use it if $\text{sample}(p)$, $p = 1/m$
- Save not used ones as well as classic output.
- In the following runs, use the previous result as well as not used L values.
- Improves the output on each run.
- Time Complexity: $O(n^3)$
- Space Complexity: $O(n \times m)$
- Runtime of first run (in Python): 0.326s



Other Approach We Tried

- Using histograms to represent the scores, instead of arrays:

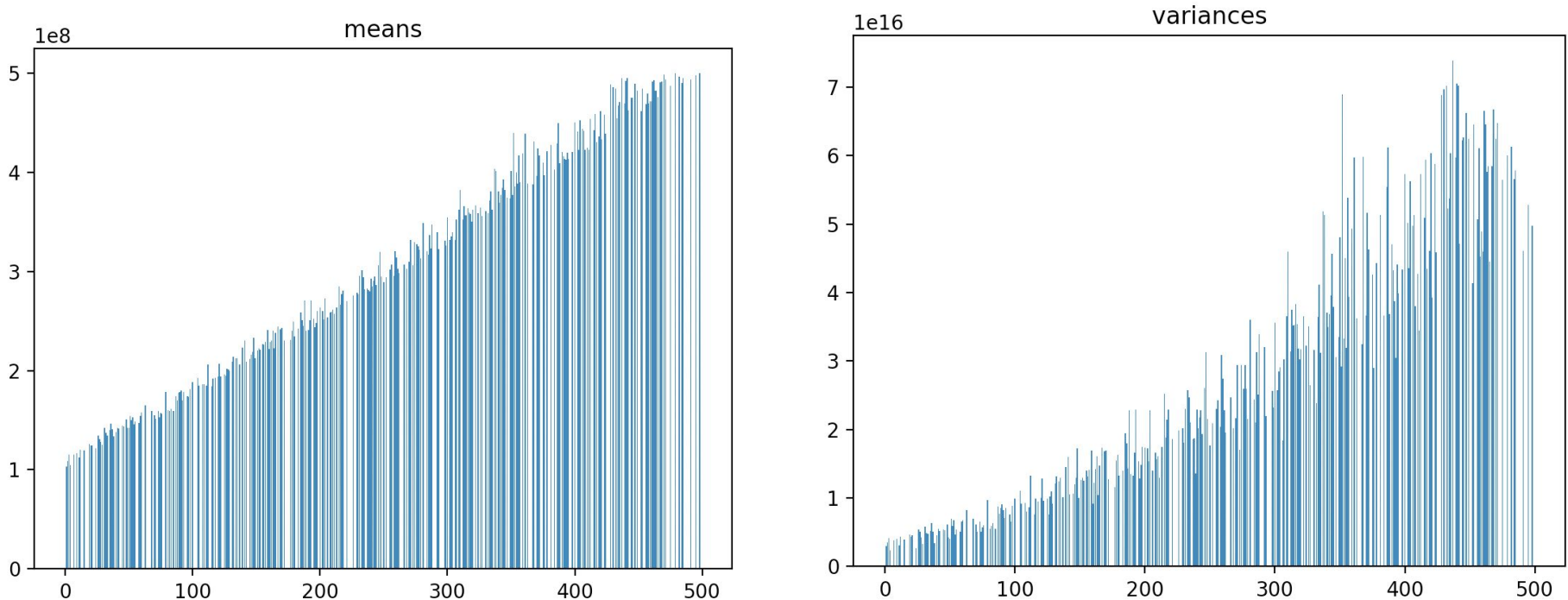
Example: [(4, 500), (6, 497),...], instead of [500, 500, 500, 500, 497, 497, 497, 497, 497, 497...]

- Reducing m by removing a value that each player had.

Example: If all players had the number x, we can remove it from all and reduce m by 1.



Results of Statistical Analysis



Some correlation found between mean and variance of players with their minimum order results. Minimum orders given on the x-axis.



Future Work

- Improvement to solution 2:
 - Instead of Insertion sort, use hybrid sorting: quicksort + insertion sort
 - Complexity would become $O(n^2 * \lg(n) * m)$.
 - Insertion sort is quick in practice though
- Use an efficient implementation of histogram with hashmaps



Questions?

Thank you for listening!